writing a machine learning classifier's core logic yourself, using pure Python, rather than importing a pre-built version from an existing library. This approach is highlighted as a significant milestone for those new to machine learning, as it demonstrates an understanding of an important piece of the puzzle.

The sources describe building two different classifiers from scratch:

• **Decision Tree Classifier (from "Let's Write a Decision Tree Classifier from Scratch - Machine Learning Recipes #8")**:

◦ This tutorial focuses on building a **decision tree classifier from scratch in pure Python**.

◦ The process involves implementing the **CART (Classification and Regression Trees) algorithm**, which is a family of algorithms used to build trees from data.

◦ Key concepts implemented from scratch include:

▪ **Deciding which questions to ask and when** to split the data.

▪ Quantifying the **uncertainty at a single node using Gini impurity**, which ranges between 0 and 1, with lower values indicating less uncertainty. It quantifies the chance of being incorrect if a label is randomly assigned.

▪ Quantifying **how much a question reduces uncertainty using information gain**. Information gain is calculated by taking the uncertainty of the starting set, subtracting the weighted average uncertainty of the child nodes after a partition.

▪ The recursive process of **building the tree**, where the best question is selected to partition the data, and the build tree function is called again on the resulting subsets until no further questions can be asked (leading to a leaf node).

◦ The goal of asking questions and splitting data is to **unmix the labels** as one proceeds down the tree, aiming for the purest possible distribution of labels at each node.

• **k-Nearest Neighbors Classifier (from "Writing Our First Classifier - Machine Learning Recipes #5")**:

◦ This tutorial demonstrates writing a **"scrappy version of k-Nearest Neighbors"** classifier from scratch.

◦ The process explicitly involves **commenting out the import of a classifier from a library** and instead implementing a custom class for the classifier.

◦ The implementation requires defining two core methods for the classifier class:

▪ **fit**: This method takes the features and labels of the training set as input. When building from scratch, this method is used to **"memorize" the training data** by storing it within the class.

▪ **predict**: This method receives the features for the testing data and returns predictions for the labels.

◦ The core logic implemented from scratch for k-Nearest Neighbors includes:

▪ **Calculating the distance** between a testing point and all training points. The **Euclidean Distance formula** is used for this, which measures the straight-line distance between two points, regardless of the number of dimensions.

▪ **Finding the closest training point** (the "nearest neighbor") to a given testing point by iterating over all training points and keeping track of the shortest distance found so far.

▪ **Predicting that the testing point has the same label as its closest training point**. For k > 1, the prediction is made based on the majority class among the 'k' closest neighbours.

In both cases, writing a classifier "from scratch" serves the purpose of **allowing you to build a simple and interpretable classifier** and gain a deeper understanding of how machine learning algorithms function at their fundamental level.